# A Pollution-Aware Urban Routing System

8th Semester Final Project

**Dhrubajyoti Mandal**
Computer Science and Engineering
KIIT University
22053859@kiit.ac.in

**Prof. Sujoy Dutta**
Computer Science and Engineering
KIIT University
sdattafcs@kiit.ac.in

## Contents

**1 Introduction** **2**

**2 System Architecture** **2**

**3 Data Collection and Preprocessing** **2**
3.1 Monitoring Station Data . . . 2
3.2 Pivoting to Wide Format . . . 3
3.3 Missing Value Imputation . . . 3

**4 Predictive Model** **3**
4.1 Problem Formulation . . . 3
4.2 Feature Engineering . . . 3
4.2.1 Lag Features . . . 3
4.2.2 Wind Direction Decomposition . . . 4
4.2.3 Cyclical Time Encodings . . . 4
4.2.4 Station Identity Encoding . . . 4
4.3 Full Feature Vector . . . 4
4.4 Model: XGBoost Regression . . . 4
4.5 Training Procedure . . . 5
4.6 Evaluation . . . 5

**5 Spatial Interpolation of $PM_{2.5}$ to Road Graph Nodes** **6**
5.1 Motivation . . . 6
5.2 Inverse Distance Weighting . . . 6
5.3 Road Type Pollution Multiplier . . . 7

**6 Route Computation** **7**
6.1 Road Graph Construction . . . 7
6.2 Composite Edge Weight . . . 8
6.3 Shortest Path Algorithm . . . 8
6.4 Turn-by-Turn Directions . . . 8

**7 Summary** **9**

**8 Demonstration** **9**

# 1. Introduction

Conventional navigation systems optimise routes with respect to travel time or distance alone. In densely populated urban environments, however, road users are continuously exposed to varying levels of ambient air pollution, primarily driven by vehicular traffic density, road classification, and meteorological conditions. This exposure is neither uniform across the road network nor static over time.

This report describes the technical design of a routing system that incorporates real-time and predicted fine particulate matter ($PM_{2.5}$) concentrations into the route cost function. Given a user-specified origin and destination, the system produces two candidate routes: a *fastest route* optimised purely on estimated travel time, and an *optimal route* that minimises a composite cost function balancing travel time against predicted pollution exposure. The system is deployed over the road network of Bhubaneswar, Odisha, India.

# 2. System Architecture

The system comprises four loosely coupled components:

1. **Data ingestion** — historical and real-time $PM_{2.5}$ readings from OpenAQ monitoring stations, retrieved via REST API and bulk S3 archive download.
2. **Predictive model** — an XGBoost regression model trained on lag features derived from historical sensor data, predicting $PM_{2.5}$ concentration 30 minutes ahead at each monitoring station.
3. **Spatial interpolation** — inverse distance weighting (IDW) to estimate $PM_{2.5}$ at arbitrary nodes in the road graph from the sparse set of monitoring station predictions.
4. **Routing engine** — a modified Dijkstra shortest-path algorithm over an OpenStreetMap-derived road graph, with edge weights derived from a composite cost function.

# 3. Data Collection and Preprocessing

## 3.1. Monitoring Station Data

Air quality data is sourced from the OpenAQ v3 API. For Bhubaneswar, the bounding box of the city's OSM relation (R10108023) is used to query all available monitoring locations:

$$\text{bbox} = (\lambda_{\text{min}},\ \phi_{\text{min}},\ \lambda_{\text{max}},\ \phi_{\text{max}})$$

where $\lambda$ and $\phi$ denote longitude and latitude respectively. Each station exposes a set of sensors, each measuring one pollutant parameter. Historical bulk data is retrieved from the OpenAQ S3 archive in CSV format, partitioned by station ID and year. Each record contains a timestamp (15-minute resolution), location coordinates, parameter name, units, and observed value.

### 3.2. Pivoting to Wide Format

Raw records are in long format: one row per (timestamp, station, parameter) tuple. Prior to feature engineering, the dataset is pivoted to wide format such that each row corresponds to a single 15-minute observation window at a given station, with each pollutant and meteorological variable occupying its own column:

$$\mathbf{x}_t = \big[\,\mathrm{pm}_{2.5},\ \mathrm{pm}_{10},\ \mathrm{wind_speed},\ \mathrm{wind_direction},\ \mathrm{temperature},\ \mathrm{rh},\ \ldots\big]_t$$

Duplicate timestamps (arising from occasional sensor overlap) are resolved by taking the mean value.

### 3.3. Missing Value Imputation

Sensors intermittently produce null readings due to instrument downtime or communication failure. Missing values are imputed per station using forward fill followed by backfill:

$$x_t \leftarrow x_{t-1} \quad \text{if } x_t = \texttt{NaN}$$

Forward fill propagates the last known valid reading; backfill handles null values at the very start of a station's history.

## 4. Predictive Model

### 4.1. Problem Formulation

The prediction task is formulated as a supervised regression problem. Given a window of $L$ historical observations at station $s$, the model predicts $\mathrm{PM}_{2.5}$ concentration $H$ steps into the future:

$$\hat{y}^{(s)}_{t+H} = f\Big(\mathbf{x}^{(s)}_t,\ \mathbf{x}^{(s)}_{t-1},\ \ldots,\ \mathbf{x}^{(s)}_{t-L+1}\Big)$$

In the deployed system, $L = 4$ (one hour of history at 15-minute resolution) and $H = 2$ (30-minute prediction horizon).

### 4.2. Feature Engineering

#### *4.2.1. Lag Features*

For each feature $x^{(k)}$ in the feature set $\mathcal{F}$, lag values are appended to the feature vector:

$$\phi_{\text{lag}} = \big\{\, x^{(k)}_{t-\ell} \ \mid\ k \in \mathcal{F},\ \ell \in \{1, 2, \ldots, L\} \,\big\}$$

This produces $|\mathcal{F}| \times L$ additional features. For $|\mathcal{F}| = 7$ and $L = 4$, this yields 28 lag features per training example.

#### *4.2.2. Wind Direction Decomposition*

Wind direction $\theta$ (in degrees) is a circular variable. Treating it as a raw scalar would imply that 1° and 359° are far apart, which is geometrically incorrect. It is therefore decomposed into its sine and cosine components:

$$\theta_{\sin} = \sin\left(\frac{\pi\,\theta}{180}\right), \qquad \theta_{\cos} = \cos\left(\frac{\pi\,\theta}{180}\right)$$

#### *4.2.3. Cyclical Time Encodings*

Hour of day $h \in [0, 24)$ and day of week $d \in \{0, \ldots, 6\}$ are likewise circular. They are encoded as:

$$h_{\sin} = \sin\left(\frac{2\pi h}{24}\right), \quad h_{\cos} = \cos\left(\frac{2\pi h}{24}\right)$$

$$d_{\sin} = \sin\left(\frac{2\pi d}{7}\right), \quad d_{\cos} = \cos\left(\frac{2\pi d}{7}\right)$$

#### *4.2.4. Station Identity Encoding*

Station identity is encoded as an ordinal integer via label encoding:

$$e_s = \text{LabelEncoder}(\text{station_id}_s)$$

This allows the model to learn station-specific pollution patterns (e.g., a busy intersection station systematically reporting higher baseline values than a residential station).

### 4.3. Full Feature Vector

The complete feature vector for a single training example is:

$$\boldsymbol{\phi} = \left[\, \phi_{\text{lag}},\ h_{\sin},\ h_{\cos},\ d_{\sin},\ d_{\cos},\ \theta_{\sin},\ \theta_{\cos},\ \text{wind_speed},\ T,\ \text{rh},\ e_s \,\right]$$

where $T$ denotes temperature and rh denotes relative humidity.

### 4.4. Model: XGBoost Regression

The model $f$ is implemented as an XGBoost gradient-boosted regression tree ensemble. Given a dataset $\{(\boldsymbol{\phi}_i, y_i)\}_{i=1}^{N}$ where $y_i = \text{PM}_{2.5}$ at time $t_i + H$, XGBoost minimises the regularised objective:

$$\mathcal{L} = \sum_{i=1}^{N} \ell(y_i,\ \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

where $\ell$ is the squared loss, $K$ is the number of trees, and $\Omega(f_k) = \gamma T_k + \frac{1}{2}\lambda \|\mathbf{w}_k\|^2$ is a regularisation term penalising tree complexity ($T_k$ = number of leaves, $\mathbf{w}_k$ = leaf weights).

The ensemble prediction is:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(\boldsymbol{\phi}_i), \quad f_k \in \mathcal{F}_{\text{trees}}$$

### 4.5. Training Procedure

The dataset is split chronologically (80% train, 20% test) without shuffling, to prevent leakage of future observations into the training set. Shuffling a time series dataset would allow the model to implicitly observe future states during training, producing artificially optimistic evaluation metrics.

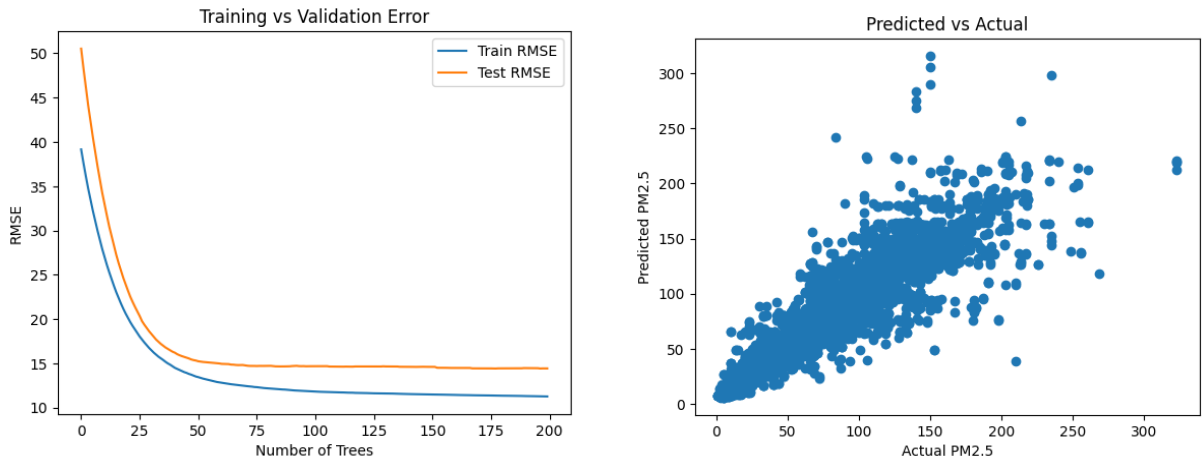
Hyperparameters used:

| Parameter | Value |
|---|---|
| `n_estimators` | 200 |
| `learning_rate` | 0.05 |
| `max_depth` | 2 |
| `subsample` | 0.8 |
| `colsample_bytree` | 0.8 |

### 4.6. Evaluation

Performance is evaluated on the held-out test set using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE):

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N} |y_i - \hat{y}_i|, \qquad \text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$




(a) Training vs Validation Error

(b) Prediceted vs Actual PM2.5 Values

Figure 1: Model Evaluation Metrics

The trained model achieved MAE = 8.55 $\mu$g/m$^3$ and RMSE = 14.43 $\mu$g/m$^3$ on the test set. Given that observed $PM_{2.5}$ values in Bhubaneswar regularly hit around 800 $\mu$g/m$^3$,

the relative MAE is approximately 1.027%, which is sufficient for meaningful route differentiation.

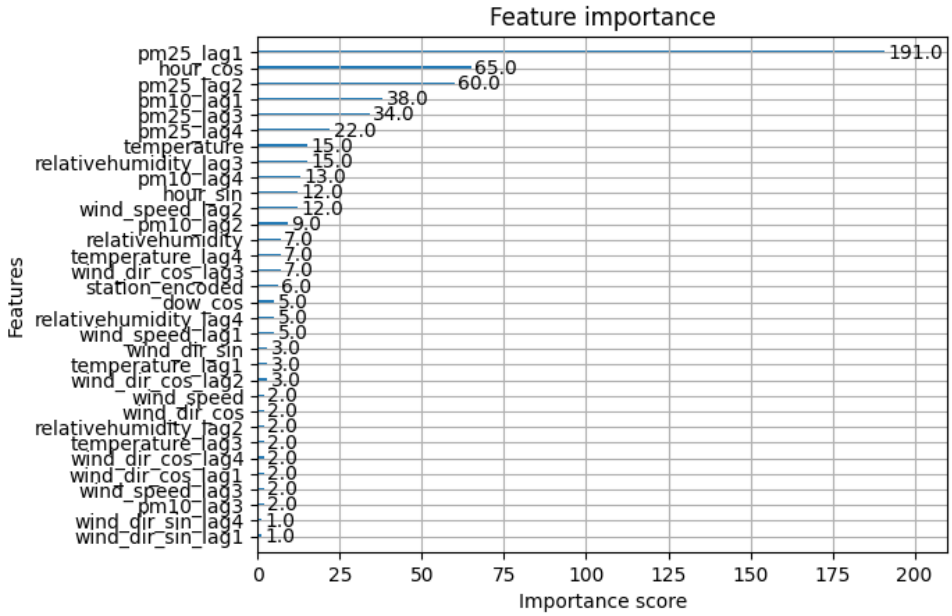



Figure 2: Feature Importance

Further, as seen in figure 2, the model relies heavily on `pm25_lag1` and the hour of day, as given by `hour_cos`.

## 5. Spatial Interpolation of $PM_{2.5}$ to Road Graph Nodes

### 5.1. Motivation

The road graph of Bhubaneswar contains tens of thousands of nodes, while only a small number of physical monitoring stations exist. To assign a pollution cost to every road segment, $PM_{2.5}$ must be estimated at arbitrary spatial coordinates. This is achieved via *inverse distance weighting* (IDW).

### 5.2. Inverse Distance Weighting

Let $\mathcal{S} = \{s_1, s_2, \ldots, s_M\}$ be the set of monitoring stations with known predicted $PM_{2.5}$ values $\{\hat{v}_1, \hat{v}_2, \ldots, \hat{v}_M\}$ and coordinates $\{(\phi_j, \lambda_j)\}_{j=1}^{M}$.

For an arbitrary road graph node $n$ at coordinates $(\phi_n, \lambda_n)$, the estimated $PM_{2.5}$ value is:

$$\hat{v}_n = \frac{\sum_{j=1}^{M} w_j \, \hat{v}_j}{\sum_{j=1}^{M} w_j}, \qquad w_j = \frac{1}{d(n,\, s_j)^2}$$

where $d(n, s_j)$ is the Euclidean distance between node $n$ and station $s_j$ in degree-space:

$$d(n,\, s_j) = \sqrt{(\phi_n - \phi_j)^2 + (\lambda_n - \lambda_j)^2}$$

This approximation is valid at city scale where the curvature of the Earth is negligible. If a node coincides exactly with a station location ($d < \varepsilon$ for small $\varepsilon$), the station's predicted value is returned directly to avoid division by zero.

### 5.3. Road Type Pollution Multiplier

IDW alone produces a smooth, nearly flat pollution surface when only two monitoring stations are available, since both cover the same city and their readings are highly correlated. To introduce spatially meaningful variation, a road-type pollution multiplier $\mu_r$ is applied:

$$\tilde{v}_{(u,v)} = \bar{v}_{(u,v)} \cdot \mu_r$$

where $\bar{v}_{(u,v)} = \frac{1}{2}(\hat{v}_u + \hat{v}_v)$ is the mean IDW estimate at the two endpoints of edge $(u, v)$, and $\mu_r$ is drawn from the following table:

| Road Type (`highway` tag) | Multiplier $\mu_r$ |
|---|---|
| `motorway` | 2.0 |
| `trunk` | 1.8 |
| `primary` | 1.5 |
| `secondary` | 1.2 |
| `tertiary` | 1.0 |
| `unclassified` | 0.9 |
| `residential` | 0.7 |

This multiplier is grounded in the well-established positive correlation between traffic volume and roadside $PM_{2.5}$ concentrations. Higher-order roads carry more vehicles and therefore generate proportionally greater pollution exposure. When live traffic data becomes available, $\mu_r$ can be replaced by a dynamic speed-based factor, leaving the rest of the pipeline unchanged.

## 6. Route Computation

### 6.1. Road Graph Construction

The road network is obtained from OpenStreetMap via the OSMnx library, queried by place name for Bhubaneswar. Static edge speeds are estimated by OSMnx from road type, and edge travel times are derived from:

$$t_{(u,v)} = \frac{\ell_{(u,v)}}{\bar{s}_{(u,v)}}$$

where $\ell_{(u,v)}$ is the edge length in metres and $\bar{s}_{(u,v)}$ is the estimated free-flow speed in m/s. The graph is modelled as a directed multigraph $G = (V, E)$.

### 6.2. Composite Edge Weight

Each edge $(u, v) \in E$ is assigned a composite weight $c_{(u,v)}$ that combines normalised travel time and normalised pollution exposure:

$$c_{(u,v)} = \alpha \cdot \tilde{t}_{(u,v)} + (1 - \alpha) \cdot \tilde{p}_{(u,v)}$$

where:

- $\tilde{t}_{(u,v)}$ is the min-max normalised travel time over all edges in $G$
- $\tilde{p}_{(u,v)}$ is the min-max normalised pollution weight (as defined in Section 5.3) over all edges in $G$
- $\alpha \in [0, 1]$ is a user-controlled trade-off parameter

Min-max normalisation maps each raw value to $[0, 1]$:

$$\tilde{x}_{(u,v)} = \frac{x_{(u,v)} - \min_{e \in E} x_e}{\max_{e \in E} x_e - \min_{e \in E} x_e}$$

This is necessary to prevent one component from dominating the other due to difference in scale (travel times are in seconds; $PM_{2.5}$ values are in $\mu g/m^3$).

**Interpretation of $\alpha$:**

- $\alpha = 1$: routing reduces to shortest travel time; pollution is ignored entirely.
- $\alpha = 0$: routing minimises pollution exposure; travel time is ignored entirely.
- $\alpha = 0.5$: equal weighting; a balanced trade-off between time and air quality.

### 6.3. Shortest Path Algorithm

Both the fastest and optimal routes are computed using Dijkstra's algorithm [1] as implemented in NetworkX:

$$\pi^* = \arg \min_{\pi \in \Pi(o,d)} \sum_{(u,v) \in \pi} w_{(u,v)}$$

where $\Pi(o, d)$ is the set of all simple paths from origin $o$ to destination $d$, and $w$ is either `travel_time` (fastest) or `composite_weight` (optimal).

### 6.4. Turn-by-Turn Directions

Once the optimal route's node sequence is determined, directions are generated by extracting *decision nodes* — nodes at which either the road name or road type changes between the incoming and outgoing edge:

$$\mathcal{D} = \left\{ v_i \mid \text{name}(v_{i-1}, v_i) \neq \text{name}(v_i, v_{i+1}) \text{ or } \text{type}(v_{i-1}, v_i) \neq \text{type}(v_i, v_{i+1}) \right\}$$

These waypoints are submitted to the OSRM map-matching endpoint (`/match/v1/driving`), which snaps the coordinate sequence to its own road network and returns structured manoeuvre instructions. Using decision nodes rather than the full node sequence substantially reduces the number of waypoints, avoiding OSRM's trace size limit while preserving routing fidelity at all meaningful junctions.

## 7. Summary

The system can be summarised as a pipeline with the following stages:

1. Retrieve real-time sensor readings for all monitoring stations in the city.
2. For each station, predict $PM_{2.5}$ 30 minutes ahead using the trained XGBoost model with lag, meteorological, and temporal features.
3. Interpolate predicted $PM_{2.5}$ values to all road graph nodes via IDW.
4. Apply road-type multipliers to compute a per-edge pollution weight.
5. Normalise travel time and pollution weights; compute composite edge weights.
6. Run Dijkstra's algorithm to obtain the fastest route (on travel time) and the optimal route (on composite weight).
7. Extract decision nodes from the optimal route and retrieve turn-by-turn directions via OSRM map matching.
8. Return both routes, their mean $PM_{2.5}$ exposure, and directions for rendering.

## 8. Demonstration

All the code can be found at: `https://github.com/BillyDoesDev/osmnx-aqi-routes`.

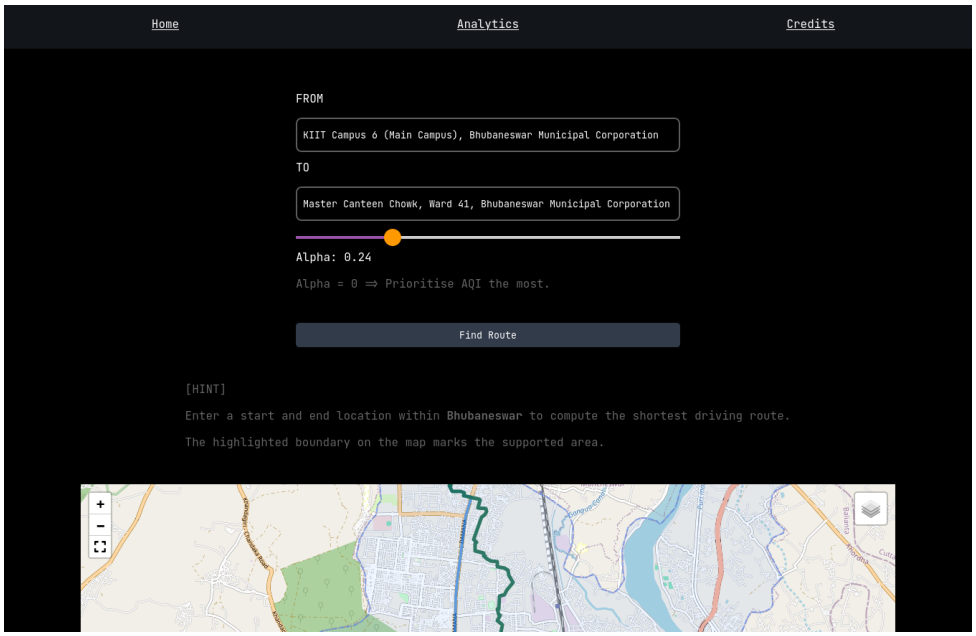



Figure 3: Project Homepage

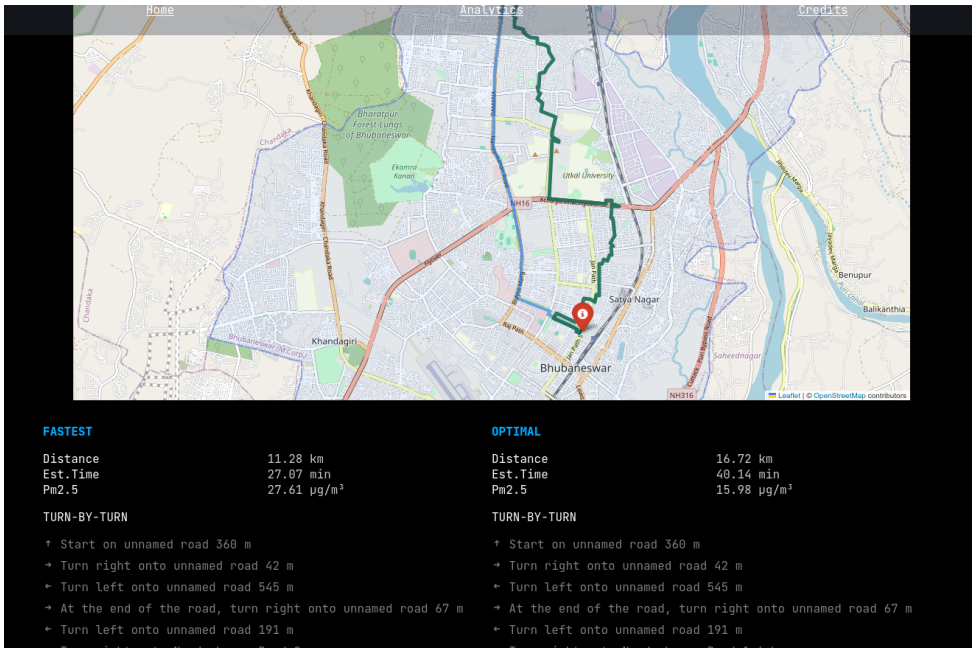



Figure 4: Project Step By Step Directions

Figure 5: Analytics

## References

[1] E.W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[3] D. Shepard, "A two-dimensional interpolation function for irregularly-spaced data," in *Proc. 23rd ACM National Conference*, 1968, pp. 517–524.

[4] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017.